Campus Venue Booking System

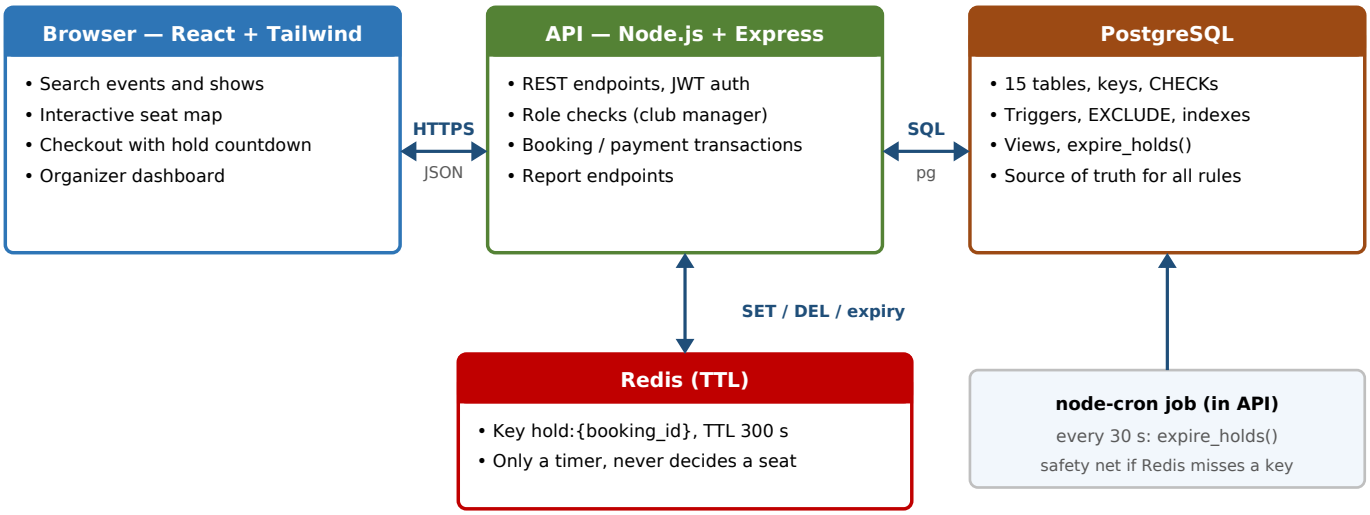
DBMS Course Project — High-Level Application Design (Mid-Sem) — Lab: P1, Group-2

Team: Areesha Shahid (2023B1A40991H), Harshitha Ambavarapu (2023B2AD0873H),
Rudhra Koul (2023B3AD0445H), Geetanjali Dangeti Sree (2023B4AD0609H), Pratyush Pranay (2023B5AA1211H)

1. Overview

A web application where students, faculty and guests browse campus events, pick exact seats (auditorium) or general-admission places (classrooms), hold them for 5 minutes, pay and receive tickets. Club managers create events, shows and price grids and view occupancy and revenue reports. The application is a thin layer: **PostgreSQL enforces the rules** (no double booking, no overlapping shows, ticket limits), and the backend starts transactions and reports errors.

2. Architecture



3. Technology stack

Layer	Technology	Role
Front end	React + Tailwind CSS	Seat map, checkout with countdown, organizer dashboard
Backend	Node.js + Express	REST API, JWT auth, transactions, role checks, report endpoints
DB driver	node-postgres (pg), plain SQL	Explicit BEGIN / COMMIT, parameterised queries
Database	PostgreSQL	Schema, constraints, triggers, views, EXCLUDE and partial indexes
Timer / cache	Redis (key TTL)	5-minute hold countdown and expiry signal
Scheduler	node-cron	Periodic expire_holds() as a safety net
Tools	Git / GitHub, Postman, pgAdmin	Version control, API and SQL testing

4. Who owns what

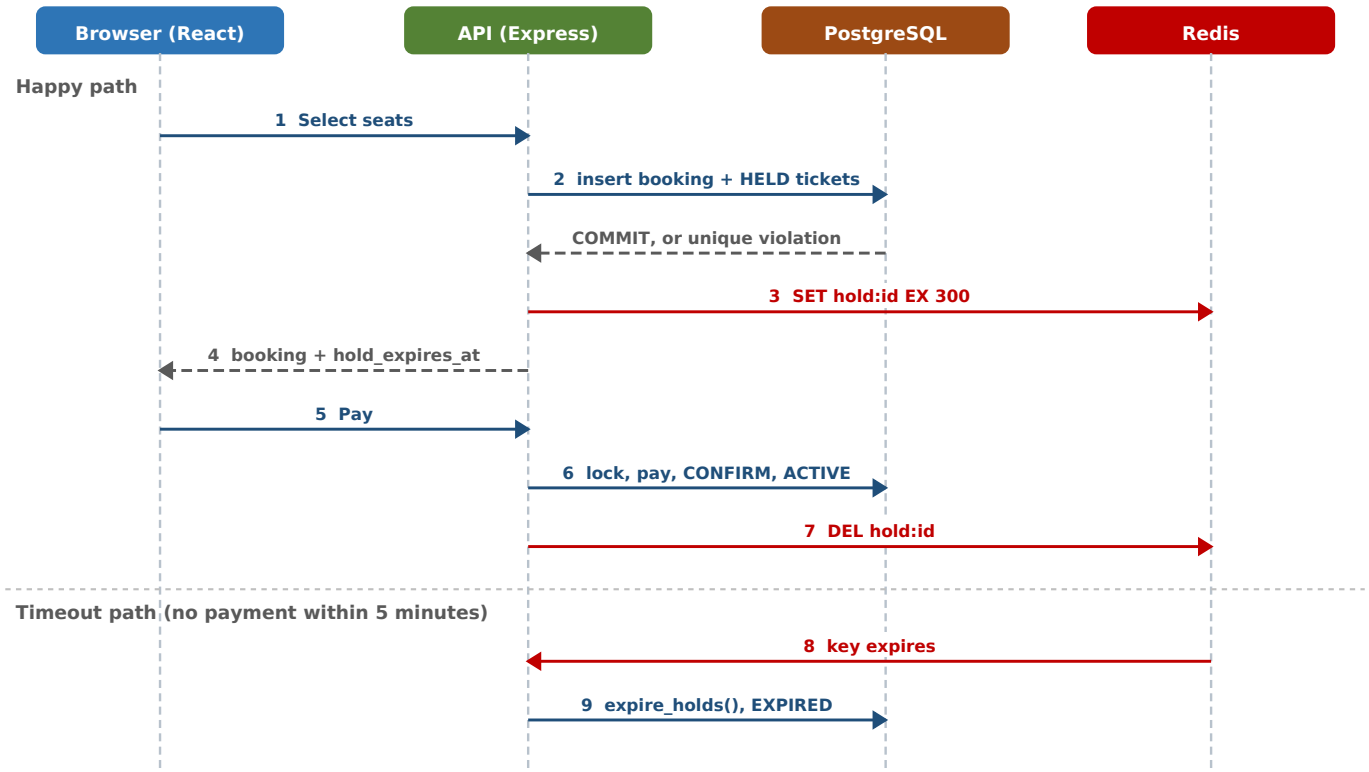
PostgreSQL — source of truth	Redis — timer only
• Decides who gets a seat (partial unique index) • Holds every booking, ticket and payment • Enforces all business rules	• Stores hold:{booking_id} with a 300 s TTL • Tells the backend when a hold has lapsed • Never decides seat ownership

If Redis is down, no data is lost and no seat can be sold twice; the cron job still expires holds.

5. Application workflows

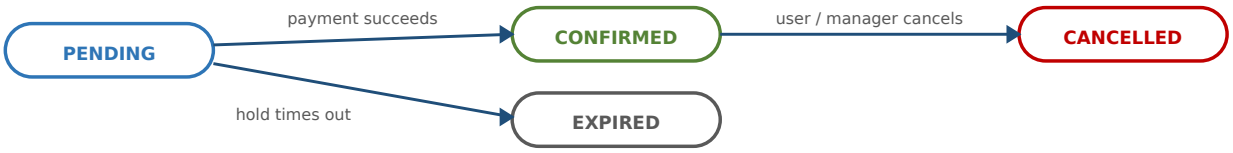
#	Workflow	Main steps	DBMS concepts
W1	Register / log in	One transaction inserts the user and its Student / Faculty / Guest row	Transaction, subclass PK = FK, triggers
W2	Hold, pay, confirm	Pick seats → hold (5 min) → pay → tickets ACTIVE (flow below)	Transaction, partial unique index, triggers
W3	Cancel	Cancel a booking, or a manager cancels a show: tickets cancelled, payments refunded	Triggers, status columns
W4	Organizer setup	Create event → show → price grid; invalid input is rejected with a clear message	EXCLUDE, CHECKs, composite FKs
W5	Dashboard	Occupancy per show, revenue by event and attendee type, top events	Joins, GROUP BY, CTE, views

6. Seat hold and payment flow (W2)



Before inserting new holds for a show, the backend runs expire_holds() for it, because an expired HELD ticket still blocks its seat until cleaned up. The countdown uses the server's hold_expires_at, not the browser clock.

Booking life cycle



7. Progress

Status	Item
Done	Requirements, assumptions and business rules
Done	ER and EER diagrams (weak entities, specialization of USER, derived and composite attributes)
Done	Relational schema (15 relations), keys, integrity constraints and design decisions
Done	Application design: architecture, tech stack, workflows, hold flow
Next	SQL scripts (tables, triggers, indexes, views) and sample data of at least 500 records
Next	Backend API, front-end screens, reports, concurrency and constraint testing

8. Team member contributions

Member	ID	Contribution
Areesha Shahid	2023B1A40991H	Requirements, assumptions and business rules; ER diagram
Harshitha Ambavarapu	2023B2AD0873H	EER diagram (weak entities, specialization of USER); ER/EER to relational mapping
Rudhra Koul	2023B3AD0445H	Relational schema: keys, composite foreign keys and integrity constraints
Geetanjali Dangeti Sree	2023B4AD0609H	Design justification; application architecture and tech stack
Pratyush Pranay	2023B5AA1211H	Application workflows and hold-and-pay flow; GitHub repository, README and documents